

Meshtastic 2.8

Nuove funzionalità di routing

Analisi tecnica approfondita del passaggio dalla linea 2.7 alla linea 2.8, con disponibilità delle funzioni per famiglia di microcontrollori.

Conclusione principale. La 2.8 non sostituisce il routing Meshtastic con un nuovo protocollo. Aggiunge quattro livelli coordinati: controllo del traffico in transito, riduzione adattiva del raggio dei broadcast, NextHop più prudente e recuperabile, autenticazione prima della modifica dello stato del router.

Base del confronto	Riferimento
Linea 2.7	upstream/2.7 - commit 2482306
Linea 2.8	upstream/develop - commit bcf486f
Versione dichiarata	2.8.0
Data dell'analisi	11 agosto 2026

Nota di certezza: al momento del confronto non esiste un ramo upstream chiamato letteralmente 2.8. La linea 2.8 analizzata è develop. Le opzioni di build e i valori indicati nel documento si riferiscono al commit riportato sopra; singole varianti possono applicare esclusioni aggiuntive.

Panoramica della nuova pipeline

In 2.8 il pacchetto viene autenticato e classificato prima che possa modificare cronologia, retry, ACK impliciti o rotte. Il Traffic Management Module interviene poi sul traffico di transito, prima del normale inoltra.

```

Pacchetto RF
|
v
Controllo autenticità e decodifica
+-- non valido -----> scarto
+-- canale sconosciuto -----> relay opaco e limitato
+-- autenticato/decodificato
    |
    v
Traffic Management Module
+-- duplicato/rate-limit -----> scarto del transito
+-- risposta NodeInfo locale --> richiesta consumata
+-- accettato -----> ReliableRouter / NextHop / flooding

```

1. Traffic Management Module

Il nuovo Traffic Management Module, o TMM, interviene sui pacchetti che il nodo dovrebbe ritrasmettere. Il suo obiettivo è evitare di spendere airtime e batteria per traffico ridondante, eccessivo o poco utile. Sui target supportati il modulo è presente e abilitato di default, ma quasi tutte le singole politiche restano opt-in.

Funzione	Stato predefinito
Deduplicazione Position	Attiva, intervallo 11 ore
Limitazione per mittente	Disattiva
Filtro pacchetti sconosciuti	Disattiva
Risposta NodeInfo per conto di terzi	Disattiva
Riduzione precisione Position	Dipende dalla precisione del canale
Esaurimento forzato degli hop	Configurato nello schema, ma non implementato
router_preserve_hops TMM	Placeholder, attualmente senza effetto

Deduplicazione delle posizioni

È la parte attiva di default. Quando arriva una posizione broadcast prodotta da un altro nodo e non destinata al nodo locale, TMM riduce la posizione alla precisione prevista dal canale e la confronta con l'ultima posizione equivalente dello stesso mittente. Se risulta sostanzialmente uguale e l'intervallo non è scaduto, il relay non la ritrasmette.

- Intervallo ordinario: 11 ore.
- TRACKER e TAK_TRACKER: intervallo massimo ridotto a un'ora.
- LOST_AND_FOUND: intervallo massimo ridotto a 15 minuti.
- position_min_interval_secs = 0 disabilita la deduplicazione.

Il confronto non conserva la coordinata completa: usa un fingerprint di 8 bit della cella geografica. È quindi una deduplicazione approssimata e molto economica in RAM, non un confronto GPS esatto. Il comportamento normale riguarda i canali pubblici o well-known.

Riduzione della precisione

Quando il canale impone una `position_precision`, il relay tronca le coordinate inoltrate alla precisione consentita. La politica viene quindi applicata al pacchetto radio, non soltanto alla visualizzazione locale. Per impostazione normale si applica ai canali pubblici o well-known; l'estensione ai canali privati è una scelta di build. La modalità radioamatoriale è esentata.

Rate limit per mittente

È disattivato finché `rate_limit_window_secs` e `rate_limit_max_packets` non sono entrambi maggiori di zero. Conta il traffico di transito per nodo sorgente: non limita i pacchetti locali né quelli indirizzati al nodo. `ROUTING_APP` e `ADMIN_APP` sono escluse per non interrompere routing essenziale e amministrazione. Il contatore ha un massimo effettivo di 60 e il tempo è quantizzato in blocchi di cinque minuti.

Filtro dei pacchetti sconosciuti

Con `unknown_packet_threshold` maggiore di zero, TMM limita per mittente i pacchetti sconosciuti in una finestra di circa cinque minuti. Non è però un filtro universale di ogni frame di canale ignoto: gli ordinari pacchetti non decifrabili vengono prima deviati dall'authentication gate nel percorso opaque relay. TMM può filtrare soltanto ciò che raggiunge il modulo.

Risposta diretta alle richieste NodeInfo

Con `nodeinfo_direct_response_max_hops` maggiore di zero, un router che possiede una NodeInfo recente e affidabile può rispondere per conto del nodo richiesto. La richiesta deve essere unicast, richiedere una risposta, non essere da o per il nodo locale, rispettare il limite di distanza e trovare un'identità realmente osservata nelle ultime sei ore.

Quando PKI è disponibile, la chiave deve essere provata tramite firma XEdDSA o verifica manuale. La risposta usa come mittente il nodo richiesto, conserva il `request_id` e imposta `hop_limit = 0`. Tre limiti contrastano la riflessione: una risposta al minuto per richiedente, una al minuto per target e almeno un secondo fra due risposte globali. Una richiesta limitata non viene eliminata: continua verso il vero destinatario.

2. Hop Scaling: raggio adattivo dei broadcast

HopScalingModule non seleziona un percorso fra A e B. Controlla dinamicamente il raggio dei broadcast periodici. Osserva soltanto nodi ricevuti realmente via LoRa, escludendo MQTT, e mantiene un istogramma di densità per distanza.

- 128 record per 512 byte complessivi.
- Hash a 16 bit del nodo, distanza da 0 a 7 hop e storico orario su 13 ore.
- Campionamento adattivo da 1 a 128 quando la rete supera la capacità della tabella.
- Persistenza in `/prefs/hopScalingState.bin`.

Calcolo della raccomandazione

Una volta all'ora il modulo stima la popolazione per distanza, accumula le fasce fino a raggiungere circa 40 nodi e può concedere un hop aggiuntivo se la popolazione rimane sotto una soglia di cortesia compresa fra 40 e 80. La soglia diventa più severa se l'attività recente cresce, intermedia su rete stabile e più generosa se l'attività cala.

Il risultato può soltanto ridurre il valore configurato dall'utente e si applica ai broadcast locali Position, Telemetry, NodeInfo e NeighborInfo. Non modifica messaggi di testo, direct message, amministrazione, traceroute o pacchetti già in transito.

- TRACKER e TAK_TRACKER mantengono almeno 2 hop.

- SENSOR mantiene almeno 1 hop.
- Dopo il boot la raccomandazione resta prudenzialmente a 7 fino alla prima elaborazione oraria valida.

Interpretazione corretta: Hop Scaling non è path routing. È una politica di contenimento del raggio e del carico radio dei broadcast periodici.

3. NextHop più sicuro nelle mesh dense

Il formato radio resta compatibile: next_hop e relay_node rimangono campi di un byte, derivati dall'ultimo byte del NodeNum. La 2.8 non elimina quindi le collisioni, ma smette di fidarsi automaticamente di un byte ambiguo.

Nuova regola conservativa: una rotta NextHop viene usata solo se il byte identifica in maniera univoca un vicino diretto plausibile. In caso contrario il nodo torna al flooding.

Apprendimento della rotta

Una rotta derivata da ACK o risposta viene accettata soltanto se il relayer è coerente con la storia del pacchetto e il suo byte si risolve in un solo nodo plausibile. L'hint viene scritto nel NodeDB e, se TMM è disponibile, anche nella sua cache di overflow, così può sopravvivere all'espulsione del destinatario dal livello caldo del database.

Route Health

La 2.8 conserva in RAM lo stato di 32 destinazioni, per circa 384 byte. Una rotta diventa non affidabile se non è confermata per 30 minuti o raggiunge tre consegne dirette fallite consecutive. Un ACK end-to-end azzerà i fallimenti e rinnova la rotta.

Quando l'ultimo tentativo diretto fallisce, il next-hop viene cancellato dal NodeDB e dalla cache TMM, quindi il tentativo finale usa il flooding. Questo impedisce che una rotta vecchia resti valida indefinitamente formando un black hole. NEXTHOP_EARLY_FLOOD_ON_UNVERIFIED esiste ma vale 0 nelle build normali.

Traceroute e favorite router

Gli hint di traceroute sono accettati soltanto quando il presunto next-hop coincide con il relay_node realmente osservato via RF. MQTT non può fornire questa prova. Anche la preservazione degli hop per i favorite router richiede ora una risoluzione univoca del byte e un ruolo Router, RouterLate o ClientBase compatibile.

Resta un limite strutturale: un altro nodo con lo stesso ultimo byte del vero next-hop può comunque interpretare il pacchetto come indirizzato a sé. Per eliminarlo completamente servirebbe ampliare il campo nel formato on-air.

4. Autenticazione prima dello stato di routing

`passesRoutingAuthGate()` classifica il pacchetto prima che possa cancellare retry, diventare ACK implicito, aggiornare `PacketHistory`, sostituire una copia accodata, modificare rotte o raggiungere moduli, telefono e MQTT.

Verdetto	Effetto
ACCEPT	Pacchetto autenticato o validamente decodificato; prosegue nella pipeline.
OPAQUE_RELAY_ONLY	Canale sconosciuto; può essere inoltrato senza interpretazione e senza mutare stato.
REJECT	Firma non valida, payload malformato o errore fatale; il pacchetto viene scartato.

Un pacchetto opaco può essere ritrasmesso soltanto in modalità ALL o ALL_SKIP_DECODING, se non è da o verso il nodo locale, ha ID valido, hop residui e next-hop assente o coincidente con il byte locale. Non aggiorna `NodeDB`, `PacketHistory`, ACK, moduli, telefono o MQTT. Le politiche XEdDSA si applicano dove tale supporto è compilato.

5. Affidabilità e prestazioni

- `PacketHistory` usa opzionalmente un indice hash open-addressing per ricerche medie $O(1)$.
- nRF52 e STM32WL escludono l'indice per risparmiare circa 1 KB e mantengono la scansione lineare.
- `ReliableRouter` rimuove lo stato di retry se l'invio iniziale fallisce, evitando retry fantasma.
- Gli invii locali annidati vengono differiti tramite una coda limitata, prevenendo rientranze e overflow dello stack.
- `NODENUM_BROADCAST_NO_LORA` viene escluso esplicitamente dalla ritrasmissione radio.
- Nelle build event mode esiste un limite hop dedicato, predefinito a 3.

6. Disponibilità per famiglia di MCU

La matrice seguente descrive le build upstream standard del commit analizzato. Core routing comprende authentication gate, opaque relay, controllo univoco del byte NextHop e Route Health. Queste correzioni fanno parte del router comune. Le esclusioni di una singola variante o MESHTASTIC_MINIMIZE_BUILD possono ridurre ulteriormente le funzioni.

Famiglia	Core routing	TMM	Hop Scaling	Hash PacketHistory	Cache NodeInfo completa
ESP32	Sì	Sì	Sì	Sì	Solo con BOARD_HAS_PSRAM
nRF52840	Sì	Sì	Sì	No	No; fallback NodeDB
RP2040 / RP2350	Sì	Sì	Sì	Sì	No; fallback NodeDB
STM32WL	Sì*	No	No	No	No
Portduino / Linux	Sì	Sì	Sì	Sì	Solo nelle unit test

* STM32WL conserva il nuovo core di routing, ma la build base esclude XEdDSA; il controllo di firma relativo a XEdDSA non è quindi disponibile salvo opt-in della variante. Restano la classificazione d'ingresso compatibile con la build, il relay opaco, le verifiche NextHop e Route Health.

Capacità dei livelli di memoria

Famiglia / configurazione	Classe	NodeDB caldo	Warm store	Cache TMM	NodeInfo payload
ESP32 classico / S2 / C3	Small	120	100	400	2000 solo se PSRAM
ESP32-S3 + PSRAM	Large	100 / 200 / 250*	2000	2048	2000
ESP32-S3 / C6 / P4 senza PSRAM	Medium	S3: 100 / 200 / 250*	150	500	-
nRF52840	Small speciale	120	100	250	-
RP2040 / RP2350	Small speciale	120	150	400	-
STM32WL	Tiny	10	0	0	-
Portduino / Linux	Large	200 predef.; runtime	2000	2048	2000 solo test

* Su ESP32-S3 il limite del NodeDB caldo dipende dalla flash rilevata al boot: 100 nodi sotto 7 MB, 200 da 7 MB, 250 da 15 MB. La cache NodeInfo payload è separata dalla cache TMM compatta: la prima conserva l'intero User per il direct-serve; la seconda usa record da 10 byte per dedup, rate limit, ruoli e hint NextHop.

ESP32

È la famiglia con il set più completo. Tutte le protezioni del core, TMM, Hop Scaling e l'indice hash di PacketHistory sono presenti nelle build standard. Con PSRAM viene compilata anche la cache di 2.000 NodeInfo complete, che rende la risposta diretta NodeInfo molto più utile sulle mesh grandi. Senza PSRAM il direct-serve resta configurabile, ma usa il NodeDB come sorgente e ha quindi una copertura inferiore. ESP32-S3 con PSRAM riceve inoltre i livelli Large: 2.048 record TMM e 2.000 record warm. Le varianti classic, S2 e C3 mantengono livelli più piccoli.

nRF52840

TMM e Hop Scaling sono presenti, ma dimensionati in modo conservativo: cache TMM da 250 record e warm store da 100. La cache NodeInfo completa non viene compilata, quindi il direct-serve usa il fallback NodeDB. PacketHistory conserva la ricerca lineare perché l'indice hash di circa 1 KB è escluso. NextHop univoco, Route Health e authentication gate restano disponibili.

RP2040 e RP2350

Ricevono core routing, TMM, Hop Scaling e indice hash. La cache TMM contiene 400 record; il warm store è aumentato a 150, ma viene deliberatamente limitato per mantenere le scritture warm.dat entro il watchdog di otto secondi. Non è presente la cache NodeInfo payload estesa, perciò il direct-serve dipende dal NodeDB.

STM32WL

È l'eccezione principale. La classe Tiny disabilita completamente TMM, la sua cache e il warm store; Hop Scaling è escluso esplicitamente. Anche l'indice hash di PacketHistory è escluso. Rimangono però le correzioni del router comune: verifica conservativa del NextHop, Route Health, fallback al flooding, protezione dell'ingresso compatibile con le primitive compilate e robustezza dei retry. La build base esclude XEdDSA e il NodeDB caldo è limitato a 10 nodi.

Portduino / Linux

Dispone di core routing, TMM, Hop Scaling, indice hash e livelli Large. Il NodeDB caldo vale 200 per impostazione normale, ma è configurabile a runtime; warm store e cache TMM arrivano rispettivamente a 2.000 e 2.048 record. La cache completa NodeInfo da 2.000 record è compilata nel percorso native soltanto per le unit test, non nella normale build Portduino.

Conseguenza pratica: dire 'Meshtastic 2.8 supporta TMM' non basta. Su STM32WL il modulo non esiste; sugli altri MCU esiste ma la profondità delle cache, la velocità di PacketHistory e la disponibilità della cache NodeInfo estesa cambiano in modo sostanziale.

7. Cosa sarà realmente attivo dopo l'aggiornamento

Senza configurazioni aggiuntive, sulle famiglie che compilano le relative funzioni si osservano soprattutto:

- deduplicazione delle posizioni ferme;
- riduzione adattiva degli hop dei broadcast periodici dopo la raccolta di dati sufficiente;
- NextHop più conservativo in presenza di byte ambigui;
- abbandono delle rotte dirette obsolete o ripetutamente fallite;
- protezione dello stato del router da pacchetti non validi o non autenticati;
- migliore scalabilità di PacketHistory sui target che mantengono l'indice hash.

Non vengono invece attivati automaticamente:

- rate limit per mittente;
- filtro TMM dei pacchetti sconosciuti;
- risposta NodeInfo per conto di terzi;
- fallback NextHop anticipato;
- esaurimento forzato degli hop configurabile in TMM, che resta un campo senza implementazione operativa.

In sintesi, la 2.8 rende il routing meno fiducioso e più recuperabile. Nelle mesh dense riduce il traffico ripetitivo, limita il raggio dei broadcast di routine e preferisce il flooding sicuro a una rotta NextHop ambigua. Nelle mesh sparse il percorso normale cambia poco, ma beneficia comunque dei controlli di autenticità e della scadenza delle rotte degradate.

Fonti tecniche

Tutti i riferimenti puntano al commit bcf486f della linea 2.8 analizzata, così il contenuto resta verificabile anche se develop continua a evolvere.

[Traffic Management Module - documentazione tecnica](#)

[NextHop direct-message reliability - analisi e mitigazioni](#)

[NodeInfo stores - livelli hot, warm e cache TMM](#)

[Implementazione HopScalingModule](#)

[Implementazione NextHopRouter e Route Health](#)

[Router core e authentication gate](#)

[Classificazione della memoria per famiglia MCU](#)

[Capacità NodeDB, warm store, TMM e feature gate](#)

Documento generato dalla risposta tecnica richiesta, con l'aggiunta della matrice di disponibilità per MCU. Le funzioni presenti nel codice ma disattivate di default sono indicate esplicitamente come tali.